DYNASTY TRADE DATABASE.

Function & Process Reference — a page-by-page walkthrough of every major capability across the four-page site, paired with a grouped index of every function in the codebase.

01

PROJECT OVERVIEW

Seven HTML pages. One brand system. Zero build step. A complete dynasty fantasy-football trade toolkit served straight from GitHub Pages.

A WHAT THIS IS

The **FantasyPoints Dynasty Trade Database** is a static, dependency-free web application that ingests live dynasty trade data, Sleeper league rosters, MVS market values, and analyst rankings to surface trade-market trends, calculate fair-value packages, audit a user's entire league portfolio, compare rankings across analysts, and document every formula on the site for analyst hand-off.

B ARCHITECTURE

Seven standalone HTML files plus a shared `assets/` tree of CSS / JS modules consumed by each page. No build pipeline, no bundler, no server-side code. Cross-page state passes through a small **session-storage handoff** envelope so a trade started on one page can be opened on another. A `data-bootstrap.js` module fetches every `data/*.json` file in parallel and fires `fpts:data-ready` when populated.

C DEPLOYMENT

The repository is served directly by **GitHub Pages** from the `main` branch. Two local-only batch scripts handle the workflow: `start.bat` opens a Python preview server, and `push.bat` stages, commits, and pushes. A push triggers a live redeploy in roughly two minutes.

D BRAND ALIGNMENT

Every page follows the **FantasyPoints Brand Style Guide v6** with a Front Office accent override — the 60 / 30 / 10 palette uses **#ED810C** dynasty orange (in place of the parent-brand red), **#111111** black, and **#F0F0F0** off-white. Kanit Extrabold Italic for display type, Mulish for body and UI copy. Both dark and light themes ship; the theme toggle is mirrored across all seven pages. Enforced by `scripts/check-colors.py` which fails any push with off-brand color drift.

02

SITE MAP

Seven pages, two directions of traffic. Every page can hand off a player, a pick, or an in-progress trade to any other page through a session-storage envelope read on load.

PRIMARY

INDEX.HTML

Dynasty Trade Database. The hub — recent trades, top players, top picks, position trends, value tracker, player detail panel.

TOOL

TRADE CALC.HTML

Trade Calculator. Build 2–3-side trades, apply league-format multipliers, see balance verdict and FAAB inclusion.

IMPORTER

MY-LEAGUES.HTML

Sleeper league importer. Auth, roster ingest, waiver analytics, archetype detection, trade suggestions, exposure rollups.

TIERS

TIERS.HTML

Dynasty Trade Tiers. Grouped or flat sortable view of every player and rookie pick, with Buy / Sell / Hold chips and month-to-month ADP comparison.

ADP

ADP-TOOL.HTML

Sleeper-derived dynasty ADP across years (2022-2026) with Box / List / Heatmap views. Dynasty Startup + Rookie source tabs; format-bucket toggles (picks / simple / rookies).

RANKINGS

RANKINGS.HTML

Two modes — Consensus (multi-analyst average with ADP overlays per format) and By Analyst (5-analyst side-by-side comparison per position with bipolar heat tints).

REFERENCE

FORMULAS.HTML

Site-wide formula + calculation catalog. 56 entries × 14 sections with provenance / examples / cross-links / "why this number" callouts. Built for analyst hand-off and developer reference.

A CROSS-PAGE HANDOFF

A pair of helpers — `_fptsWriteHandoff(obj, targetPage)` on the sending side and `_fptsReadHandoff()` on the receiving side — serialize an envelope into `sessionStorage`, open the target page in a new tab, and let the receiver hydrate its UI from the same data. Every page exposes named entry points (the `_fptsXpage*` family) so the hand-off origin is auditable.

B DATA SOURCES

Embedded — `KTC_DATA` (KeepTradeCut values, position rank, age, ADP), `ADP_DATA`,

Live — Sleeper public API for users, leagues, rosters, projections, transactions, drafts, traded

03

TRADE DATABASE

index.html — the hub of the application. Every other page connects back here. Five primary tabs, a live player detail panel, and an embedded trade finder.

RECENT TRADES	Default landing tab. Filtered, sortable card view of the entire trade corpus. Expand any card to see assets side-by-side with values, league format chips, and the trade's snapshot date.
TOP PLAYERS	Most-traded players ranked by appearance count. Click a row to see every trade that included that player. Position chips inline on each row.
TOP PICKS	Same surface as Top Players, but for draft capital — every **Year + Round** combination ranked by trade frequency, expanded inline.
POSITIONS	Position-level breakdown of trade volume — QB / RB / WR / TE / PICKS with expansion behavior matching the other lists.
VALUE TRACKER	Watch-list-style view of league-rank badges for the user's roster — total rank, player-only rank, position ranks, pick-capital rank.

SECONDARY SURFACES

Player detail panel — opens on player click. Articles, photo, Sleeper-driven bio (years of experience, height, team, status), stats tab, age-curve chart, and an embedded Trade Finder seeded with the player.

Pick detail panel — opens on pick click. Mirrors the player panel surface for a specific year/round, with its own Trade Finder.

Two-sided trade search — top-bar tool. Add players or picks to Side A, Side B, and the corpus is filtered to trades where both sides match together. Drives the entire results set, not just one tab.

Format filters — QB count (1QB/SF), starter count, team size, TEP, PPR, PPC, FAAB on/off, passing TD, asset cap, roster size, and date range with presets (7/30/90 days, year, all-time).

04

LEAGUE IMPORTER

my-leagues.html — type in a Sleeper username and the page does the rest. Multi-league ingest, per-league archetype detection, league-wide trade suggestions, and cross-league exposure rollups.

AUTH & INGEST	Search by Sleeper username → load every league for the selected season → fetch rosters, users, traded picks, drafts, transactions, and league-history chain in parallel.
LEAGUE ROWS	One row per league. Sortable by league name, season, format, total value, archetype. Expand a row to see the full league dashboard.
LEAGUE DASHBOARD	Per-league: position-rank pills, full roster with KTC values + Sleeper projections, optimal-lineup highlighter, owned-picks list with computed pick values, standings, waivers, trade history (multi-year chain), and a team-by-team comparison view.
ARCHETYPES	Each team is classified as **Dynasty**, **Contender**, **Tweener**, **Rebuilder**, or **Emergency** from a 9-cell mapping over (value vs. league median) × (age vs. league median).
TRADE SUGGESTIONS	For any opposing player or pick: generate 1/2/3-asset packages from the user's roster that hit the target's KTC value within a tolerance band, score them by value-fit + archetype-fit + asset-count preference, and surface diverse picks (fair value, slight overpay, archetype-tilted).
CALC MODAL	In-page trade calculator scoped to a specific league — search assets within that league only, computed totals and balance, "open in standalone calculator" handoff.
WAIVERS	Four sub-tabs: leaderboard (most successful bidders), top bids (largest FAAB outlays), recent activity, by-team rollup. All driven by transaction history across every regular-season week.
EXPOSURE	Aggregates the user's holdings across every loaded league. One row per player and pick, sortable by share count, exposure %, position rank, or KTC value. Click into any row to see the contributing leagues.

05

DYNASTY TIERS

tiers.html — the hand-tuned ranking layer. Every player and every rookie pick is bucketed S++ through C- (TAT's 12-tier value ladder), with editorial signals (Buy/Sell/Hold, Priority, Contender) on every row.

GROUPED VIEW	Default. Players collapsed under tier headers in canonical tier order (S++, S+, S, A+ … C-). Each tier header shows the editorial description for that tier.
FLAT VIEW	Single sortable table. Click any column header to sort: Tier, Player, Age, Pos, Pos Rank, Team, 2026 ADP, 2025 ADP, Auction, 2025 PPG, Trending.
FILTERS	Position filter (All / QB / RB / WR / TE / PICK) and tier filter (any single tier). Filters apply identically across both views.
EDITORIAL CHIPS	**Buy/Sell/Hold** — Buying, Checking In, Selling, Shopping, Temp Hold. **Priority** tier for must-target players. **Contender** flag for win-now value.
CROSS-PAGE	Click a player name to open it inside the main Trade Database (`index.html`) with the player detail panel pre-loaded. Toolbar buttons hand off the page to the Database or Calculator.

06

TRADE CALCULATOR

trade-calculator.html — pure builder. Two sides by default, expandable to three. Format-aware multipliers, FAAB inclusion, and a balance verdict that updates on every keystroke.

SIDES	Side A and Side B at minimum. Add a third side for three-team trade modeling. Each side has its own search input, asset list, FAAB field, and total readout.
FORMAT AWARE	Header controls — QB count, PPR, TEP, passing TD value — apply a per-position multiplier to every player on the board. QB gains in SF, TE gains in TEP, RB/WR shift with PPR. Pick values pass through untouched.
FAAB	Dollars are folded into the side total at a 10× weighting — comparable scale to KTC values for a quick "is the FAAB enough?" read.
BALANCE BAR	Live verdict: `Fair Trade` if within 5%, otherwise the larger side wins by a percentage. Color-coded green / yellow / red. Difference value shown beneath.
TRADE-SEARCH PANEL	Same two-sided trade search that lives on the Database — drop assets into Side A and Side B to see real trades where both sides appeared together.
HANDOFFS	Open the current trade in the Database (filtered to matching real trades), or open it in My Leagues (to compare against the user's actual rosters).

07

KEY USER FLOWS

Three end-to-end journeys that demonstrate how the four pages compose into a single decision-making workflow.

FLOW 1 — COMP A REAL TRADE

"Is this trade fair?"

1. Open **index.html** → Recent Trades.
2. Click "+ Add" on the trade-search bar. Drop the proposed Side A and Side B assets in.
3. Inspect the filtered cards — every real trade in the corpus where both sides appeared together.
4. Click **Open in Calculator ↗** to model the same trade in the standalone calculator with format multipliers applied.

FLOW 2 — ROSTER AUDIT

"Where do I stand across all my leagues?"

1. Open **my-leagues.html**, enter Sleeper username, choose season.
2. Each league row shows total value, archetype, and a position-rank pill bar. Sort by archetype to see all Contender leagues together.
3. Expand a league → review roster, optimal lineup highlight, owned picks with computed value, standings, waivers.
4. Scroll to the Exposure section to see which players the user has the highest league-share in — natural sell/hold candidates.

FLOW 3 — BUILD A COUNTER-OFFER

"What can I send for their guy?"

1. In a league dashboard, click any opposing player or pick → **Suggest Trade**.
2. Page generates 1-, 2-, and 3-asset packages from the user's roster scored on value-fit + archetype-fit.
3. Walk through templates with ✓ (accept) / ✗ (next). ✓ opens the league-scoped calc modal with the package pre-loaded.
4. Tune the package and hand it off to the standalone calculator for a clean read.

08

FUNCTION REFERENCE.

Every major function across the four files, documented one card at a time. Helpers and one-line utilities are grouped at the end of each section so the headline functions stay scannable.

A HOW TO READ THIS SECTION

Each major function gets its own card with name, signature, a plain-language description of what it does, and (where relevant) the key collaborators it calls. Helpers — pure formatters, predicates, theme glue — are listed in compact grids at the close of each file's section.

B SECTION ORDER

09 — index.html (Trade Database) · 10–12 — my-leagues.html (League Importer) · 13 — tiers.html · 14 — trade-calculator.html.

C CONVENTION

Names prefixed with an underscore (`_drawTradeFinder`) are internal collaborators, not entry points. Names prefixed with `_fptsXpage` are cross-page handoff entry points. `ml*` functions live in `my-leagues.html` ; `pp*` functions are player-panel utilities; `tf*` functions belong to the Trade Finder.

09

INDEX · FILTERS & TRADE SEARCH

The filter pipeline and two-sided trade search are the central nervous system of the Database. Every tab renders off the same `filtered` *array.*

FILTER PIPELINE

applyFilters CORE

applyFilters()

Reads every filter control's current value (QB count, starters, teams, TEP, PPR, PPC, FAAB, passing TD, asset cap, roster cap, date range) and narrows the master `TRADES` array into `filtered`. Two-sided trade search runs in the same pass — each side's assets must all appear together in the same trade. Updates the filter-chip bar, the result count, and calls `renderAll()`.

CALLS GETSELECTEDPICKS · RENDERALL

applyPreset UI

applyPreset(type)

One-click format presets — SF, 12-team, half-PPR, etc. Sets the underlying filter selects to the preset's values and runs `applyFilters()`.

setDatePreset · clearDates UI

setDatePreset(days) · clearDates()

Date-range shortcuts. `setDatePreset(7)` fills from/to inputs with today minus 7 days through today, then refilters. `clearDates()` blanks both inputs.

resetFilters UI

resetFilters()

Returns every filter control to "all", clears pick selections and trade-search lists, then refilters.

TWO-SIDED TRADE SEARCH

tradeSearchInput SEARCH

tradeSearchInput(side, val)

Live search-as-you-type on Side A or Side B of the trade search. Matches against players first (capped at six), then rookie picks. Renders the dropdown with focused-row keyboard hints.

tradeSearchKey · tradeSearchAdd · removeTradeSearchAsset SEARCH

tradeSearchKey(e, side) · tradeSearchAdd(side, label, type, key, pos) · removeTradeSearchAsset(side, key)

Keyboard navigation (Arrow keys / Enter), commit a selection into the side's asset list, and remove an asset. Each commit triggers `applyFilters()`.

renderTradeSideTags · renderAssetFilterTags RENDER

renderTradeSideTags(side) · renderAssetFilterTags()

10

INDEX · TABS & RENDERING

Every tab renders off the same filtered corpus. The expand/collapse pattern is shared across players, picks, and asset types.

renderAll · showTab RENDER

renderAll() · showTab(name)

Master render — re-runs the active tab's render function. `showTab()` swaps the active tab, mirrors the tab strip state, and calls `renderAll()`.

renderRecent TAB

renderRecent()

Recent Trades tab. Renders the filtered trades as cards in reverse-chronological order using `tradeCardHtml()`. Falls back to a "no results" notice when filters exclude everything.

renderTopPlayers · renderTopPicks · renderPositions TAB

renderTopPlayers() · renderTopPicks() · renderPositions()

The three "most-traded" rollups. Each aggregates the filtered trade list into the appropriate buckets (player name, year+round, position), sorts by count, and renders an expandable list.

renderValueTracker TAB

renderValueTracker()

Value Tracker tab. Renders the user's tracked roster with league-rank badges (total, players-only, per position, picks-only). Each row uses the same player row template via the local `renderRow()` closure.

toggleExpandPlayer · toggleExpandPick · toggleExpandAssetType · _mtpCollapseAll EXPAND

toggleExpandPlayer(name, ev) · toggleExpandPick(year, round, ev) · toggleExpandAssetType(type, ev) · _mtpCollapseAll()

The shared expand/collapse mechanism for "most-traded" lists. Stores the currently-expanded key on `window.MTP_EXPANDED` so the same row toggles cleanly on a second click. Inline-renders the matching trades below the clicked row.

tradeCardHtml RENDER

tradeCardHtml(t)

The canonical trade-card template. Builds the side-by-side layout, asset rows with thumbs and value, league format chips, and the snapshot date. The inner `renderAssets(side)` closure handles each side.

getAllAssets DATA

getAllAssets()

Returns the union of every player in `KTC_DATA` plus every active rookie pick, normalized into a single shape: `{ label, type, value, posKey }`. Powers search, the Trade Finder, and the player-panel finder.

11

INDEX · PLAYER PANEL

The right-hand panel that opens on player or pick click. Five tabs: Trades, Player Stats (click any year for week-by-week + playoff breakdown), Age Curve, Trade Finder, ADP Heatmap. Articles render as a banner inside the panel hero (not a tab).

openPanel · openPanelContent · closePanel PANEL

openPanel(playerName) · openPanelContent(playerName) · closePanel()

openPanel() drives the open transition and delegates the content render to **openPanelContent()**, which resolves the player's KTC entry, Sleeper ID, photo, articles, and bio (height, team, years of experience, status), then mounts the active tab.

ppShowTab PANEL

ppShowTab(tabName)

Switch the active panel tab. Lazy-renders Stats, Age Curve, and Finder content the first time their tab is shown, so the initial open is fast.

ppSearchInput · ppSearchKey · ppSearchSelect · ppSwitchTo · ppRemovePlayer · ppRenderTabStrip · ppFocusResult

PANEL SEARCH

ppSearchInput(val) · ppSearchKey(e) · ppSearchSelect(name) · ppSwitchTo(name) · ppRemovePlayer(name)

The panel-local search box and tab strip. Lets the user open multiple players in adjacent panel tabs, navigate between them, and remove tabs without closing the whole panel.

renderPlayerStats · _fetchWeeklyStats · _renderWeeklyRowsHtml · _toggleWeeklyExpand

STATS

renderPlayerStats(containerId, player) · _fetchWeeklyStats(sleeperId, year) · _toggleWeeklyExpand(yearCell, sleeperId, year, cols, colCount, capturedName)

Fetches season stats from Sleeper for the player's Sleeper ID and renders a position-aware stats table — one row per year with FPTS / GP / position-specific counting stats / PPG. Each Year cell is **clickable**: a small chevron (▸ ▾) toggles an inline expansion that fetches weekly stats (regular + post-season in parallel via **grouping=week**) and inserts child rows below the year row using the same column structure. Playoff weeks marked with a bright orange "PLAYOFF" chip. Per-(player, year) results cached in **_weeklyStatsCache** for the page session. Falls back gracefully when no Sleeper ID is mapped.

ASYNC-GUARD PATTERN PER CLAUDE.MD §1: EVERY FETCH CAPTURES THE TARGET PLAYER AT CALL TIME AND BAILS AT RESOLUTION IF THE USER HAS SWAPPED TO A DIFFERENT COMPARED PLAYER. INLINE-STYLE CONSTANTS _THBASE · THSTYLE · THSTYLENAME · _TDBASE · TDSTYLE · TDNUM · TDNAME REUSED BETWEEN THE SEASON TABLE AND WEEKLY CHILD ROWS SO CELLS ALIGN VERTICALLY.

renderAgeCurve CHART

renderAgeCurve(containerId, player)

Renders the player's projected value-vs-age curve as inline SVG. Uses per-position curve presets (peak age, peak value, ramp-up exponent, decay rate) and scales them to the player's current KTC value. Overlays the player's current

12

INDEX · FINDER & HELPERS

The Trade Finder is an embedded calculator that lives inside the player and pick panels. Closes out with the pick panel and helper grids.

TRADE FINDER

renderTradeFinder · _drawTradeFinder FINDER

renderTradeFinder(containerId, seedAsset) · _drawTradeFinder(containerId)

Mount the Finder inside any container and seed Side A with the panel's active asset. **_drawTradeFinder()** is the redraw loop — every state change re-runs it to redraw both sides, the balance bar, and the active add-slot dropdown.

tfGetState · tfActivateSlot · tfDeactivateSlot · tfQuerySlot · tfShowSuggestions · tfAddFromSearch · tfRemove

FINDER

tfQuerySlot(containerId, side, val) · tfAddFromSearch(containerId, side, label, value, posKey, type) · tfRemove(containerId, side, idx)

Per-container Finder state (Side A list, Side B list, active query). Activate/deactivate the search slot, type into it, commit a search result, or pop an asset off a side.

getSuggestions LOGIC

getSuggestions(targetGap, existingSide, containerId)

Suggests assets to balance an unbalanced Finder. Filters out assets already on the board, then returns the eight whose value is closest to the gap.

PICK PANEL

openPickPage · closePickPanel · closPickPage PANEL

openPickPage(year, round) · closePickPanel()

Opens a panel scoped to a specific year + round draft pick. Mirrors the player panel surface: trades the pick appeared in, embedded Trade Finder seeded with the pick. **closPickPage** is a legacy alias.

pkpShowTab PANEL

pkpShowTab(name)

Tab switcher inside the pick panel — same role as **ppShowTab()** on the player side.

HELPERS · INDEX.HTML

FORMATTING

fmtDate(d) · human-readable trade date
highlightMatch(name, query) · wrap search hits in <mark>
_imgThumb(name, size) · player avatar w/ initials fallback

UI GLUE

toggleMobileFilters() · show/hide filter drawer
clearMainSearch() · empty the top-bar search
toggleTheme() · dark ↔ light, persisted

CROSS-PAGE HANDOFFS

_fptsXpageOpenCalcFromPanel() · → trade-calculator.html
_fptsXpageOpenLeaguesFromPanel() · → my-leagues.html
_fptsXpageOpenCalcFromFinder() · finder → calculator

13

MY-LEAGUES · AUTH & INGEST

Sleeper username in → every league, every roster, every traded pick, every transaction out. The ingest pipeline is fan-out parallel against the public Sleeper API.

apiFetch NETWORK

```
async apiFetch(url)
```

Single fetch wrapper. JSON-parses on 2xx, returns `null` on any error. Used by every Sleeper-facing call so failures don't break the parent `Promise.all()` fan-outs.

mlSearch · getPlayers · loadAndRenderLeagues · loadLeaguesForSeason AUTH/INGEST

```
async mlSearch() · async getPlayers() · async loadAndRenderLeagues() · async loadLeaguesForSeason(year)
```

Top-level auth and league discovery. Resolves the username → `user_id`, primes the Sleeper player dictionary, pulls every league for the chosen season, and renders the league list. Excludes leagues with `status === 'complete'` by default.

fetchAllLeaguesData INGEST

```
async fetchAllLeaguesData()
```

Fan-out fetch for every league in the active season — pulls rosters, users, traded picks, and draft chain in parallel and stashes the result under `window.ML_ALL_LEAGUE_DATA[leagueId]`. Surfaces the count of leagues where the user is a co-commish but not an owner (those skip roster-scoped pulls).

getLeagueFormat HELPER

```
getLeagueFormat(l)
```

Derives a one-line format label ("12-Team Dynasty SF PPR TEP") from the league's `roster_positions`, `total_rosters`, and `scoring_settings`.

mlSignOut AUTH

```
mlSignOut()
```

Clear cached username + all loaded league data and return to the search-by-username screen.

14

MY-LEAGUES · LEAGUE LIST RENDERING

One row per league with archetype, value totals, and position-rank pills. Click to expand into the full per-league dashboard.

renderLeagueRows · mlSetLeagueSort RENDER

renderLeagueRows() · mlSetLeagueSort(key)

Renders one row per league with the format chip, total value, archetype, and a position-rank pill bar. Sort by league name, total value, or archetype (a custom canonical order, not alphabetical).

toggleLeagueExpand · ensureLeagueDataLoaded EXPAND

async toggleLeagueExpand(leagueId) · async ensureLeagueDataLoaded(leagueId)

Expand/collapse a league row. **ensureLeagueDataLoaded()** is the lazy gate — it pulls the per-league data the first time only, then renders the dashboard on every subsequent expansion.

renderExpandedLeagueContent RENDER

renderExpandedLeagueContent(leagueId, bodyEl)

Renders the full per-league dashboard: position-rank pills, roster section, owned picks, league-team comparison, standings, waivers, trade history.

15

MY-LEAGUES · VALUE MATH

Per-league value math, position ranks, and archetype detection — the data layer behind every league dashboard.

mlComputeLeagueValueData CORE

```
mlComputeLeagueValueData(leagueId)
```

The heart of the league dashboard. Per team: sum KTC value by position, compute pick value from owned picks, derive average age, then rank every team across the league. Returns the user's own ranks (TOTAL, PLAYER, PICK, QB/RB/WR/TE) plus the league archetype medians used downstream.

CALLS MLGETOWNEDPICKS · MLPICKVALUE · MLGETARCHETYPE

mlGetArchetype · mlArchetypeLabel · mlArchetypeChip · mlArchetypeBg · mlArchetypeFg

ARCHETYPE

```
mlGetArchetype(avgAge, totalValue, leagueAvg)
```

9-cell classifier: high/low/mid value × young/old/mid age → Dynasty, Contender, Tweener, Rebuilder, or Emergency. The label/chip/bg/fg helpers are pure presentation glue.

mlPickValue · mlGetOwnedPicks PICKS

```
mlPickValue(season, round) · mlGetOwnedPicks(rosterId, tradedPicks, leagueRounds, currentSeason)
```

Pick value lookup (KTC table by season and round) and the resolver that walks `tradedPicks` to determine which roster currently owns each of the next ~3 seasons' picks.

16

MY-LEAGUES · ROSTER & OPTIMAL LINEUP

Roster ingest, the big render, the optimal-lineup solver, and the team-by-team comparison view.

loadRoster · loadRosterScoped ROSTER

async loadRoster(leagueId, season) · async loadRosterScoped(leagueId, season, scopeKey)

Loads the active roster for the user in a specific league. Pulls weekly projections (or season projections in pre-season), the current matchup week, and traded picks. The *scoped* variant caches under a custom key so multiple league expansions don't trip over each other.

renderRoster RENDER

renderRoster(playerIds, players, starters, myPicks, rosterToTeam, projections, currentWeek, leagueStatus)

The big render — roster grouped by position, value & trend per player, projected points, optimal-lineup highlights, and a picks section showing owned picks with computed value. Heavy use of inner closures (`playerRow` , `posSection` , `picksSection` , `pickValue`) to keep the per-row HTML tight.

_mlComputeOptimalLineup LOGIC

_mlComputeOptimalLineup(playerIds, players, projections, rosterPositions)

Solves the optimal lineup. Ranks every player by projection (or KTC value pre-season), then walks roster slots in precedence order (locked positions before flexes before SUPER_FLEX), assigning the highest-scoring eligible player to each slot. Returns the set of optimal-starter IDs so the renderer can highlight mismatches against the user's set lineup.

mlSortLeagueTeams · renderLeagueTeamsSection · mlToggleTeamRoster · mlBuildTeamRosterHtml

COMPARE

renderLeagueTeamsSection(leagueId, ld, sortKey) · mlToggleTeamRoster(leagueId, rosterId) · mlBuildTeamRosterHtml(leagueId, rosterId)

The team-by-team comparison view. Renders one row per opponent with their archetype, total value, position-rank colored badges, and a click-to-expand roster that lazy-builds the first time it's opened.

17

MY-LEAGUES · TRADE SUGGESTIONS

Generate counter-offer packages from the user's roster, score by value-fit and archetype-fit, and walk them one card at a time.

mlGenerateTradeSuggestions LOGIC

`mlGenerateTradeSuggestions(myAssets, targetValue, archetype, maxCount)`

The package generator. Filters out tiny assets, then enumerates 1-, 2-, and 3-asset combinations whose total lands inside a [0.85, 1.30] × target band. Each candidate is scored on value-fit (55%) + archetype-fit (35%) + fewer-assets preference (10%). The output is bucketed for variety — fair value 1-asset, fair value 2-asset, slight overpay, value package, archetype-tilted — and the rest filled by top score.

CALLS MLPACKAGEFIT · MLFITNOTE

mlBuildAssetPool · mlPackageFit · mlFitNote LOGIC

`mlBuildAssetPool(leagueId, rosterId) · mlPackageFit(pkg, archetype) · mlFitNote(pkg, archetype, fitScore)`

Build the user's tradeable asset pool for a specific league (players + picks normalized to a common shape). The fit functions score and describe how well a candidate package matches the opposing team's archetype (Contender wants veterans, Rebuilder wants picks and youth, etc.).

openTradeSuggestModal · openTradeSuggestModalForPick · closeTradeSuggestModal MODAL

`openTradeSuggestModal(sleeperId, leagueId) · openTradeSuggestModalForPick(season, round, leagueId, ownerRosterId)`

Open the one-card-at-a-time pre-screener seeded with either a player or a pick. Builds asset pools for both sides, runs the generator, and stashes the templates inside the module-scoped `MLTB` state.

mltbRender · mltbReject · mltbAccept · mltbOpenCalculatorBlank · mltbCalcCtaBlock MODAL

`mltbRender() · mltbReject() · mltbAccept(payload) · mltbOpenCalculatorBlank()`

The walkthrough UI. Show the current template, ✗ advances, ✓ opens the league-scoped calc modal with the package pre-loaded. After all templates are reviewed the user can open the calculator blank to design their own.

18

MY-LEAGUES · CALC MODAL & WAIVERS

League-scoped in-page calculator and the four waiver sub-tabs, both driven by the same per-week transaction ingest.

CALC MODAL

openCalcModal · closeCalcModal · renderCalcModal · mlCalcSideTotal · mlCalcSearch · mlCalcAdd · mlCalcRemove · mlCalcHideResults

MODAL

```
openCalcModal(targetName, targetSleeperId, opts) · renderCalcModal()
```

In-page calculator scoped to a league — the search index includes only that league's players plus the user's owned picks. Two sides, totals on every change, balance verdict. Doubles as the "open with pre-loaded package" target for the trade suggest flow.

WAIVERS

fetchAndCacheWaivers · mlEnsureWaiversRaw INGEST

```
async fetchAndCacheWaivers(leagueId, players, displayWeek) · async mlEnsureWaiversRaw(leagueId)
```

Pulls every week's transactions in parallel and aggregates them per player ID — count of adds and average winning bid. The raw waiver list is also retained for the sub-tabs that need transaction-level detail.

mlWaiversSetSubtab · renderWaiversSubtab · renderWaiversLeaderboard · renderWaiversTopBids · renderWaiversRecent · renderWaiversByTeam

RENDER

```
mlWaiversSetSubtab(leagueId, subtab, tabEl) · renderWaiversSubtab(leagueId, subtab)
```

Four waiver sub-tabs — Leaderboard (most successful bidders), Top Bids (largest FAAB outlays), Recent (full activity feed), By Team (per-team pickup rollup). Each receives the shared raw data and renders independently.

19

MY-LEAGUES · HISTORY, EXPOSURE, AVAILABILITY

Standings with positional scoring breakdowns, multi-year trade history walking the league-chain, exposure across every loaded league, and cross-league availability for any single player.

STANDINGS & PICKS

loadStandings · renderStandingsTable · mlSetStandingsSort STANDINGS

```
async loadStandings(leagueId) · renderStandingsTable() · mlSetStandingsSort(key)
```

Pulls all rosters, users, and up to 18 weeks of matchup data; derives wins/losses/PF/PA, max points (raw `fpts_max`), per-position scoring breakdown, archetype composite (60% player value + 20% pick value + 20% projection), and **per-position Max PF contribution (MPX %)** via an optimal-lineup simulation that walks `league.roster_positions` and greedily fills each starter slot with the highest-scoring eligible player. Renders a sortable table (Total Value / Archetype / Avg Age / Record / Points For) plus four "Position Rankings" cards with Value Rank, Pts Rank, and MPX % stat blocks. The user's row is highlighted.

WIRED UP 2026-05-17 (SECOND SESSION) AS A 3RD TAB NEXT TO TRADE HISTORY / WAIVERS VIA A `DATA-MODE="STANDINGS"` BRANCH IN `MLHISTORYSETMODE`. WAS PREVIOUSLY DEAD CODE (FUNCTION EXISTED BUT NO UI BUTTON INVOKED IT). THE HANDLER INJECTS THE EXPECTED DOM SCAFFOLDING (`#STANDINGS-CONTENT · #STANDINGS-LOADING · #STANDINGS-SECTION-META`) INTO THE SHARED `TRADES-CONTENT-{LEAGUEID}` CONTAINER BEFORE CALLING `LOADSTANDINGS(LEAGUEID)`. PER-POSITION MPX% FORMULA DOCUMENTED AT DOCS/FORMULAS.MD §57.

loadPicks PICKS

```
async loadPicks(leagueId)
```

Loads the user's owned-picks list for the picks-panel surface — every pick the user owns in this league, sorted, with computed KTC value.

TRADE HISTORY (MULTI-YEAR)

loadAllTrades · loadAllTradesScoped · showTradeYear · renderTradeYear · mlHistorySetMode

HISTORY

```
async loadAllTrades(leagueId, season) · showTradeYear(leagueId, idx, tabEl) · async renderTradeYear(leagueId, idx, allTradesByYear, players, leagueChain)
```

Walks the Sleeper league-chain (`previous_league_id`) back as far as 2017, fetching trades for every season. `renderTradeYear()` renders the active year's trades; `mlHistorySetMode()` swaps between three modes — Trade History, Waivers, and Standings (Standings wired in 2026-05-17 second session via a new `data-mode="standings"` branch). All three share the same `trades-content-{leagueId}` content container; the mode handler injects mode-specific scaffolding and dispatches to `renderTradeYear` / `renderWaiversSubtab` / `loadStandings` respectively.

CROSS-LEAGUE AVAILABILITY

mlGetPlayerAvailability · renderPlayerAvailability AVAILABILITY

```
mlGetPlayerAvailability(sleeperId) · renderPlayerAvailability(sleeperId)
```

For a given player, scans every loaded league to determine: *mine*, *owned by X*, *free agent*, or *no roster*. Used inside the player-detail modal to surface "owned by you in League A, free in League B, owned by Smith in League C" rollups.

20

MY-LEAGUES · DETAIL & HELPERS

The cross-league player detail modal pulls it all together — your status in every league, owner, archetype, KTC value, articles. Closes out the file with helper grids.

openPlayerDetail · closePlayerDetail MODAL

openPlayerDetail(sleeperId) · closePlayerDetail()

Opens the player detail modal. Aggregates the per-league status rows (`mlGetPlayerAvailability`), the player's KTC value, photo, years of experience, status, and the cross-league exposure summary. Mounts the article feed via the shared `mountPlayerArticles()` .

mlGetPlayerStatus · mlGetKtcByName · mlPlayerName RESOLVE

mlGetPlayerStatus(sleeperId) · mlGetKtcByName(name) · mlPlayerName(sleeperId)

Quick resolvers used by every row: status (*mine*, *owned by X*, *free*) with FAAB summary, KTC entry by canonicalized name, and full display name from Sleeper player ID.

UI TOGGLES & PERSISTENCE

AVAILABILITY SECTION

mlGetAvailabilityCollapsed() · read collapsed state
mlSetAvailabilityCollapsed(on) · write collapsed state
mlToggleAvailability() · toggle & persist
mlGetHideUsernames() · privacy toggle
mlSetHideUsernames(on) · privacy toggle

EXPOSURE SECTION

mlGetExposureCollapsed() · read collapsed state
mlSetExposureCollapsed(on) · write collapsed state
mlApplyExposureCollapsed() · enforce state on render
mlToggleExposure() · toggle & persist

HELPERS · MY-LEAGUES.HTML

FORMATTING & CHIPS

mlEscapeHTML(s) · XSS-safe text
posColor(pos) · position → hex
posBadge(pos) · position pill HTML
rankBadge(rank) · numeric rank chip
fmtDate(ts) · unix → readable
_mlPosrNum(posR, fallback) · parse position rank

AVATARS & UI

playerThumb(sleeperId) · Sleeper avatar img
pickThumb() · pick placeholder
mlImgInitialsFallback(img) · broken-img fallback
showError · hideError · global error banner
showSpinner · hideSpinner · loader toggle
findRosterIdByUser(map, userId) · reverse lookup

CROSS-PAGE HANDOFFS

_fptsXpageMlPdToDb() · player → database
_fptsXpageMlPdToCalc() · player → calculator
_fptsXpageMlTsToDb() · suggest target → db
_fptsXpageMlPepToDb() · pick exposure → db
_mlPepGoToCalc(...) · pick exp → calc
toggleTheme() · dark ↔ light

21

TIERS · GROUPED & FLAT VIEWS

A small, tight file. Two views, three editorial chip columns, and a hand-curated tier order driving both group rendering and sort.

RENDER & SORT

renderTiers · setView RENDER

renderTiers() · setView(v)

Main renderer. In *grouped* mode, walks `TIER_ORDER` and emits one section per tier with its description header. In *flat* mode, applies the active sort and renders a single sortable table. `setView()` swaps modes and updates the toggle button state.

applyFilters FILTER

applyFilters(players)

Applies the position filter (All / QB / RB / WR / TE / PICK) and tier filter (any single tier) to the source list. `PICK` uses `isPick()` rather than a position match.

setSortCol · sortPlayers SORT

setSortCol(col) · sortPlayers(players)

Sort only applies in flat view. `setSortCol()` toggles direction on the same column or resets to ascending on a new column. `sortPlayers()` handles tier ordering through `TIER_ORDER.indexOf()`, numeric columns via `parseFloat`, and auction by stripping non-digits before parsing.

renderRowHtml · renderHeaderRow RENDER

renderRowHtml(p) · renderHeaderRow()

The row and header templates. Header columns: Tier · Player · Age · Pos · Pos Rank · Team · 2026 ADP · 2025 ADP · Auction · 2025 PPG · Trending · Buy/Sell · Priority · Contender.

CHIPS & HELPERS

EDITORIAL CHIPS

bshChipHtml(val) · Buy / Sell / Hold
priorityChipHtml(val) · must-target tier
contenderChipHtml(val) · win-now flag
tierBadgeClass(tier) · S++…F- → CSS class

PREDICATES & FORMAT

pc(p) · position → CSS class
isPick(p) · rookie-pick detector
fmtCell(v) · empty-safe text cell
fmtNumeric(v) · empty-safe number cell
toggleTheme() · dark ↔ light

CROSS-PAGE HANDOFFS

FROM TOOLBAR

_fptsXpageTiersToDb() · → index.html
_fptsXpageTiersToCalc() · → trade-calculator.html

FROM ROW

_fptsTiersOpenPlayer(name) · player click → db w/ panel open

22

TRADE CALCULATOR · BUILDER & BALANCE

Pure builder. Format-aware multipliers, FAAB inclusion at 10× weight, and a balance verdict that updates on every input event.

BUILD & RENDER

renderAll · renderBuilder RENDER

renderAll() · renderBuilder()

Top-level render — composes the builder columns and then recalculates. `renderBuilder()` emits one side card per entry in `sides` : side header with total, asset list, FAAB row, and the side's own search input + results dropdown.

recalc · renderBalance CALC

recalc() · renderBalance()

Recalc total readouts on each side, then redraw the balance bar. For two-side trades the verdict is `⚖ Fair Trade` under 5% difference, otherwise `▲ Side X wins (+N%)` with a color-coded fill.

sideTotal · adjVal · getMultiplier LOGIC

sideTotal(side) · adjVal(asset) · getMultiplier(pos)

`sideTotal` sums adjusted asset values plus FAAB×10. `adjVal` applies the position multiplier (picks pass through untouched). `getMultiplier` reads the format selects and returns the per-position factor — QB up 15% in SF, RB shifts 0.93–1.04 by PPR, TE adds 12% per TEP point, etc.

SIDES & ASSETS

addSide · removeSide · updateFaab · resetAll STATE

addSide() · removeSide(id) · updateFaab(sideId, val) · resetAll()

Add a third side (max three), remove a side, update a side's FAAB allotment, or reset the entire builder. Every state change re-runs `renderAll()` .

searchAssets · _doSearch · showResults · hideResults · searchKeydown · addAssetToSide · removeAsset

SEARCH

searchAssets(sideId, query) · addAssetToSide(sideId, asset) · removeAsset(sideId, idx)

Per-side asset search. Splits matches into players + rookie picks, renders a dropdown of up to ~10 results, supports keyboard escape to close, and pushes/pops the side's asset list with re-renders triggered after each mutation.

FILTERS & TRADE-SEARCH (SHARED WITH DATABASE)

applyFilters · applyPreset · resetFilters · setDatePreset · clearDates · tradeSearchInput · tradeSearchKey · tradeSearchAdd · removeTradeSearchAsset

FILTERS

The same filter and two-sided trade-search engine as `index.html` . The Calculator embeds it so the user can comp their built trade against real trades without leaving the page.

23

TRADE CALCULATOR · HELPERS & HANDOFFS

Shared utilities, UI glue, and the cross-page handoff entry points that move a built trade into the Database or My Leagues.

AVATARS & PREDICATES	UI GLUE	HANDOFFS
_imgThumb(name, size) **pickThumb(size)** **normalizePlayerName(name)** **getAllAssets()** **highlightMatch(name, query)**	**renderPickTags()** **renderTradeSideTags(side)** **renderAssetFilterTags()** **clearMainSearch()** **toggleMobileFilters()** **toggleTheme()**	**_fptsXpageCalcToDb()** · → index.html **_fptsXpageCalcToLeagues()** · → my-leagues.html **_fptsCalcCurrentTrade()** · serialize builder state

24

DATA PIPELINE

How player data flows from source CSVs and APIs into every page's UI — layer by layer, with MVS as the canonical value source.

A SEVEN JSON FILES IN DATA/

Each page bootstraps by fetching: `data/values.json` (FP + Sleeper player metadata, written by `sync-fp.py`), `data/adp.json` (dynasty ADP buckets — startup variants `picks/simple/rookies` × SF/1QB, plus rookie-only drafts `rookie_draft_sf/rookie_draft_1qb`, plus the legacy unified `rookie` key — from the sleeper_dynasty_adp pipeline, written by `sync-adp.py`), `data/auction.json` (Sleeper auction medians, same source), `data/pick-availability.json` (per-entity 14×12 availability heatmap matrices; 300 real players + ~77 ROOKIE_PICK_X.YY rookie pick slots), `data/picks.json` (pick values), `data/articles.json` (FantasyPoints article mentions), and `data/mvs.json` (canonical Market Value Score, written by `sync-mvs.py` from `data/source/player_market_mvs.csv`).

Lockstep guarantee: `adp.json`, `auction.json`, and `pick-availability.json` are written in the same `sync-adp.py` invocation with identical `version` timestamps (single `now` value). The heatmap's "Data refreshed" stat cell pulls from that version field, so it always matches the ADP refresh date wherever the heatmap renders.

A2 PICKS BUCKET: REAL PLAYERS + RDP PLACEHOLDERS

In dynasty startups that pre-draft rookie picks as Kicker placeholders, `sync-adp.py`'s `classify_startup_drafts` identifies the fingerprint (K in rounds 1..4) and `relabel_picks_K_to_rdp` rewrites those K `player_id` values to `ROOKIE_PICK_X.YY` before aggregation. The resulting `picks_sf` view has real top-of-board players (rank 1: Josh Allen, ADP 2.6) intermixed with rookie pick slots (rank 16: Rookie Pick 1.01, ADP 16.0). `build_rdp_pick_availability` applies the same trick to produce a heatmap matrix for each rookie pick slot; both functions share `_availability_matrix_from_picks` as their math kernel.

A3 ROOKIE-ONLY DRAFTS: SF/1QB SPLIT

After the primary `build_adp` aggregation, sync-adp.py runs a second pass that duplicates rows where `dynasty_class == 'rookie'` and re-tags the duplicates as `rookie_draft_sf` or `rookie_draft_1qb` based on `is_superflex`. The legacy unified `rookie` view-key is preserved untouched for backward compat (my-leagues / DB still read it). On the ADP Tool, the new keys feed the Dynasty Rookie ADP top-level tab. Sample sizes for 2026: `rookie_draft_sf` = 420 records (Jeremiyah Love at ADP 1.0, 8,449 drafts); `rookie_draft_1qb` = 114 records.

A3B SITE-WIDE HEADSHOT-FALLBACK RULE

25

LEGEND SYSTEM

Per-page expandable documentation drawer — keeps developers and curious users oriented as the site grows.

A THREE SHARED FILES

`assets/css/legend.css` styles the trigger button + slide-out drawer + accordion sections. `assets/js/legend.js` handles open/close/render. `assets/js/legend-content.js` holds the per-page content dictionary keyed by page slug (`index` / `adp-tool` / `trade-calculator` / `my-leagues` / `tiers`).

B ITEM SCHEMA

Every item carries a **label** (UI name), **what** (1-2 sentence description), **source** (file/function/path, monospaced), **values** (how to interpret displayed numbers), and optional **notes**. Algorithm entries opt into five additional optional fields rendered in narrative order: **inputs** (what feeds the algo), **formula** (the actual math, monospaced), **output** (how to read the result), **example** (a worked example with real numbers, rendered with an orange left-border accent block), and **codeRef** (file:line for navigation). Missing fields gracefully omit from the rendered output, so simple UI entries can stay terse while algorithm entries get full developer-grade depth.

C WIRING A PAGE

Each page links `assets/css/legend.css` in its head, then loads `legend-content.js` + `legend.js` in order. On DOMContentLoaded the page calls `Legend.init('page-key')` which builds the floating trigger + drawer DOM and binds keyboard + click handlers. Escape and outside-click both close the drawer.

D COVERAGE TODAY

~195 items across 41 sections, 5 pages. Trade Database is the deepest (~75 items including a new "Data Pipeline" section that documents the MVS overlay precedence). Six dev-grade algorithm entries use the full schema: Archetype Scoring (my-leagues), Picks-as-Assets Relabeling (adp-tool), Trade Finder (index), Sleeper API Coupling (my-leagues), MVS Overlay Precedence (index), and Cross-Page Handoff (index). Each page ends with an **"Interactions & Conventions"** quick-reference section covering position colors, brand red, keyboard shortcuts, mobile widths, localStorage keys, and numeric thresholds.

SHARED MODULES & PERSISTENCE

Two patterns that keep the site coherent: shared JS modules in `assets/js/`, and localStorage keys that cross page boundaries.

A DATA-BOOTSTRAP.JS (THE NEW DATA LAYER FOR FUTURE PAGES)

`assets/js/data-bootstrap.js` is the single shared data layer for pages built with the scaffold. Set `window.FPTS_CURRENT_PAGE` before the script tag, then it fetches all 7 data/*.json files in a single `Promise.all`, populates 9 `window.*` globals (FP_VALUES, PICK_VALUES, SLEEPER_IDS, ADP_PAYLOAD, AUCTION_PAYLOAD, MVS_PAYLOAD, PLAYER_ARTICLES, PICK_AVAILABILITY, PICK_AVAILABILITY_META), runs the canonical `_applyAdpPayload` / `_applyAuctionPayload` / `_applyMvsPayload` helpers, and fires `document.dispatchEvent('fpts:data-ready')` when done. The existing 5 pages still inline their bootstrap; they migrate to this when next touched.

A2 BRAND.CSS (CANONICAL VISUAL RULES)

`assets/css/brand.css` holds the brand variables (`--red` = #ED810C, full position palette, dark/light tokens), self-hosted Kanit + Mulish font faces, top-nav styles, page chrome (`.page` , `.page-header` , `.page-title` , `.page-subtitle`), and shared widget styles (`.pos-pill` , `.team-pill` , `.stat-pill` , `.trend` , `.icon-btn`). Linked by the scaffold; existing pages still inline their copies. Pulling brand-level styles out makes new pages a fraction of the size and guarantees consistent typography + colors across the site.

A3 PAGE-TEMPLATE.HTML (THE SCAFFOLD)

`templates/page-template.html` is the copy-this-when-starting-a-new-page file. Has the full `<head>` with all the shared module includes wired correctly, the canonical top-nav block, the three required mount divs (`#pp-mvs-extras` , `#pp-articles-mount` , `#pp-league-context`), and a `fpts:data-ready` listener skeleton at the bottom. Three clearly-marked TODO spots: title, FPTS_CURRENT_PAGE slug, and the `<main id="page-body">` content. Full flow documented in `docs/WORKFLOW.md` § "2b. Add a new page or tool".

A4 TEAM-HELPERS.JS (NFL TEAM LOGOS)

`assets/js/team-helpers.js` exposes `window.TeamHelpers` with `logoUrl(team)` , `logoImg(team, opts)` , `headshotBadge(team, opts)` , and `wrapWithBadge(headshotHtml, team, opts)` . Backed by Sleeper's CDN at `sleepercdn.com/images/team_logos/nfl/{team.toLowerCase()}.png` — same origin as player headshots, no auth, transparent-background PNGs. The site-wide convention for every trade-chip / player-list surface is to emit `logoImg(team, { size: 18 })` directly to the right of the player name span; parent flex container uses `align-items: center` for baseline. Plain-text fallback when `window.TeamHelpers` isn't

27

CLOSING NOTES

A presentation reminder, plus the working conventions that will keep this codebase coherent as it grows.

DO'S & DON'TS

✓ Add new feature work as one self-contained block per HTML file — inline CSS, inline JS, inline data.	✗ Introduce a build step, bundler, or external JS dependency.
✓ Pass cross-page state through the `_fptsWriteHandoff` / `_fptsReadHandoff` envelope.	✗ Drop state into URL hashes or query strings without an envelope schema.
✓ Reuse `getAllAssets()` for any new "search players or picks" feature.	✗ Re-derive the asset list from `KTC_DATA` in a new helper — keep one source of truth.
✓ Escape user input through `mlEscapeHTML()` before insertion into innerHTML.	✗ String-concatenate Sleeper-returned text directly into innerHTML.
✓ Keep headlines in Kanit Extrabold Italic, body in Mulish — matches Brand Style Guide v6.	✗ Set body copy in Kanit or headlines in Mulish.
✓ Use the brand palette — `#ED810C`, `#111111`, `#F0F0F0` — through CSS variables.	✗ Inline arbitrary hex values into per-component styles.
✓ When adding or changing UI features, update `assets/js/legend-content.js` for the affected page slug.	✗ Ship a new feature without documenting it in the legend — developers handed the codebase will look there first.
✓ Layer the MVS overlay on top of `FP_VALUES` via `_applyMvsPayload` — it preserves Sleeper-sourced metadata.	✗ Read player values from `data/values.json` directly without applying the MVS overlay — values will be stale.
✓ Keep the 5-tab strip **identical** across every player modal: Trades / Player Stats / Age Curve / Trade Finder / ADP Heatmap. Update all four (DB / Calc / My-Leagues / ADP-tool) together.	✗ Add or remove a tab on one modal without mirroring the change across the other three. Push.bat warns; user-facing inconsistency is unacceptable.
✓ Keep asset-chip rendering aligned: Trade Calculator `.asset-row` and Trade Finder `.tf-asset` share the same position palette + stacked first/last name layout.	✗ Change the chip palette or layout on one builder without mirroring on the other. Push.bat counts the data-pos rules per side and warns on drift.